Kazimara — Architecture technique V1

Oct 26, 2026 · @Soso

Prestataire : NumuConnect, représenté par Sokhna FALL
Client : Kazimara, projet porté par M. Thiouna Ndiaye
Version : 1.0 — document d'architecture technique

1. Objet et périmètre

Kazimara V1 = une app mobile iOS + Android, une app web avec page d'accueil, un backend unique, et un canal WhatsApp avec saisie vocale en français et en wolof. Ce document fixe l'architecture technique à suivre pour la développer. Il remplace la partie technique des documents JOKKO (cahier des charges, fiche technique, business plan) là où ils divergent.

Inclus en V1

Domaine	Ce que fait la V1
Ventes et facturation	Déclarer une vente, émettre une facture conforme Sénégal, la partager en PDF
Encaissements	Espèces, Wave, Orange Money, Free Money via PayDunya ; paiement partiel ou total
Ardoise (crédit client)	Suivre ce que chaque client doit, relancer par WhatsApp
Stock	Catalogue produits de la boutique, stock décrémenté à chaque vente, alerte de stock bas
Compta simplifiée	Trois flux : ventes, encaissements, dépenses → livre de caisse automatique
Tableau de bord	Ventes du jour et du mois, encaissé, reste à encaisser, stock bas
WhatsApp + vocal	Déclarer une vente par message texte ou vocal (français, wolof), confirmée d'un tap
Connexion	Numéro de téléphone + code SMS, puis PIN ou empreinte

Domaine	Ce que fait la V1
Langues	Interface en français et en anglais

Reporté en V2 ou plus tard : plusieurs boutiques et plusieurs vendeurs par commerce, rôles fins, RH et paie, CRM et campagnes, marketplace B2B, financement embarqué, certification fiscale DGID, imprimante ticket et scan de code-barres, autres pays que le Sénégal.

Hypothèses de dimensionnement : une boutique = un utilisateur en V1, environ 1 800 commerces payants la première année, Sénégal uniquement, monnaie FCFA, TVA 18 %.

2. Principes et stack retenue

Tout est en TypeScript, dans un seul dépôt, avec un backend en monolithe modulaire. Une petite équipe (un ou deux développeurs) peut ainsi tenir le mobile, le web et le serveur sans changer de langage.

Principes directeurs

1. **Le serveur fait foi.** Les montants (HT, TVA, TTC), le statut des factures et les paiements sont recalculés et validés côté serveur. Le téléphone affiche, le serveur décide.
2. **Hors-ligne d'abord sur le mobile.** Déclarer une vente, consulter le stock et la compta marchent sans réseau.
3. **Rien d'irréversible sans confirmation.** Une vente dictée en vocal n'est jamais enregistrée sans validation explicite du commerçant.
4. **Portabilité.** Tout tourne en conteneurs Docker, sans service propriétaire bloquant, pour pouvoir migrer de l'Allemagne vers un datacenter sénégalais.
5. **Léger pour les petits téléphones et la 3G.** Peu de données échangées, images compressées, synchronisation par petits lots.

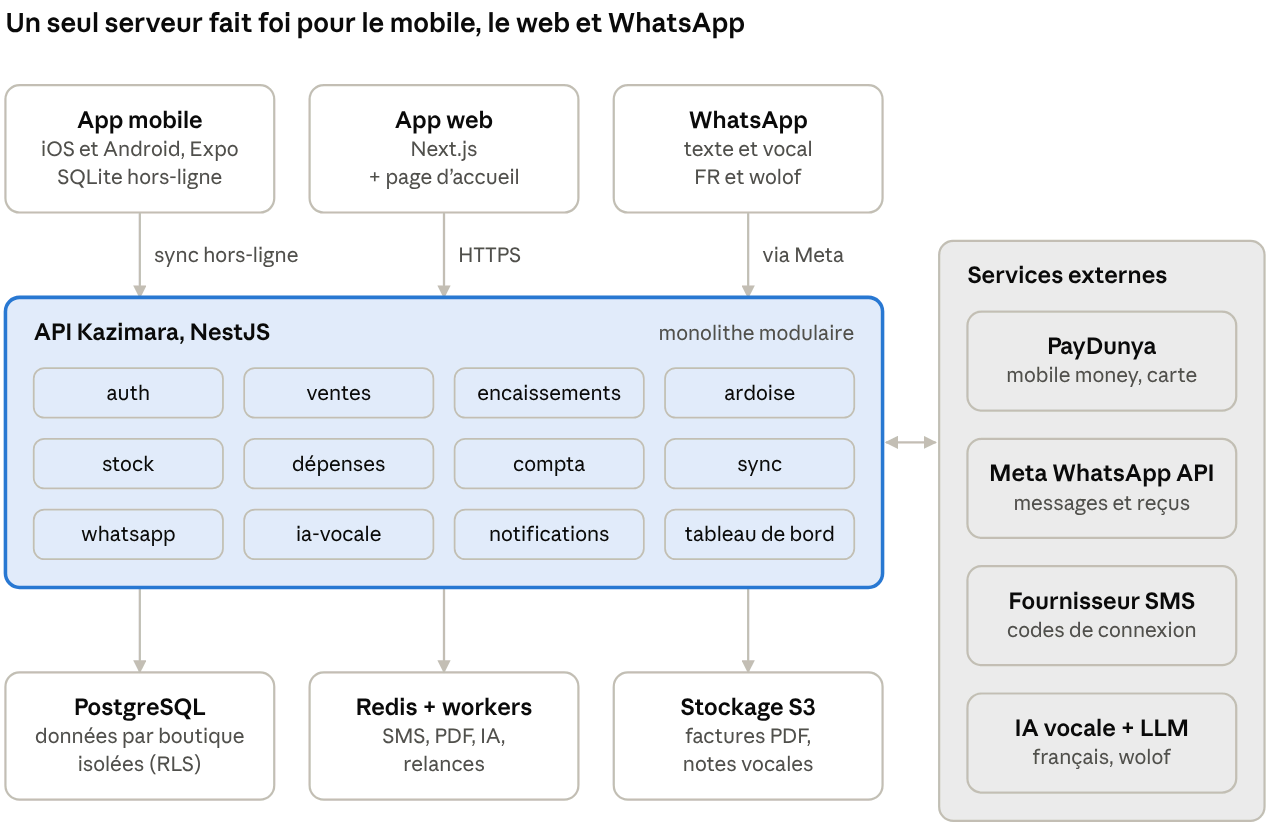
Couche	Technologie	Rôle
App mobile	React Native + Expo (SDK récent), TypeScript	iOS et Android depuis un seul code ; builds et mises à jour via EAS
Base locale mobile	SQLite (expo-sqlite) + Drizzle ORM	Données hors-ligne et file de synchronisation

Couche	Technologie	Rôle
App web + accueil	Next.js (App Router), TypeScript	Page d'accueil publique et application web connectée
UI web	Tailwind CSS + composants maison aux couleurs Kazimara	Charte vert baobab, terre cuite, or soleil
Backend	NestJS, TypeScript	API REST, modules métier, webhooks
Base de données	PostgreSQL 16+	Données transactionnelles, isolation par boutique
ORM serveur	Drizzle ORM ou Prisma	Schéma typé, migrations versionnées
Tâches asynchrones	Redis + BullMQ	SMS, PDF, WhatsApp, IA vocale, relances
Fichiers	Stockage objet S3-compatible	Factures PDF, notes vocales
Validation partagée	Zod	Mêmes règles sur mobile, web et serveur
Paiement	PayDunya (agrégateur)	Wave, Orange Money, Free Money, carte
Messagerie	WhatsApp Business Cloud API (Meta)	Reçus, relances, saisie conversationnelle
IA vocale	Reconnaissance vocale + modèle de langage	Transcrire puis extraire la vente
Suivi	Sentry + logs structurés	Erreurs mobiles et serveur

Flutter et Supabase, cités dans les documents JOKKO, ne sont pas retenus. Flutter imposerait un second langage (Dart). Supabase disperserait la logique métier et compliquerait l'auto-hébergement.

3. Vue d'ensemble

Les trois canaux (app mobile, app web, WhatsApp) parlent tous à la même API. Elle seule écrit dans la base, appelle les services externes et reçoit leurs notifications.



architecture Kazimara V1 · 3 canaux, 1 API, 4 services externes

L'app mobile garde une copie locale pour travailler sans réseau et se synchronise dès qu'elle peut. Les tâches lentes (SMS, PDF, transcription vocale, relances) passent par une file Redis pour que l'API réponde toujours vite.

4. Organisation du code

Un seul dépôt Git (monorepo pnpm + Turborepo) contient les trois applications et les paquets partagés. Une règle de calcul (TVA, total d'une vente) est écrite une fois et utilisée partout.

```
kazimara/
  apps/
    mobile/        React Native + Expo (iOS, Android)
    web/           Next.js : page d'accueil + app web
    api/           NestJS : API, webhooks, workers
  packages/
    domain/        règles métier pures (calcul TVA, totaux, statuts)
    schemas/       schémas Zod partagés (entrées/sorties d'API)
    api-client/    client HTTP typé généré depuis OpenAPI
    i18n/          traductions FR / EN
    ui-tokens/     couleurs, typographies, espacements Kazimara
  infra/
    docker/        Dockerfiles, docker-compose de dev
    deploy/        configuration des environnements
```

Modules du backend (un dossier par module, chacun avec ses tables et ses routes ; aucun module ne lit directement les tables d'un autre) :

Module	Responsabilité
auth	Numéro + OTP SMS, sessions, appareils
boutiques	Fiche commerce (raison sociale, NINEA, RCCM, adresse), paramètres
catalogue	Produits, prix, unités, seuils d'alerte
ventes	Ventes, lignes, factures, numérotation, PDF
encaissements	Paiements (espèces, PayDunya), webhooks IPN
ardoise	Soldes clients, échéances, relances
stock	Mouvements de stock, alertes
depenses	Saisie des dépenses et catégories
compta	Livre de caisse calculé à partir des ventes, encaissements et dépenses
sync	Réception des lots hors-ligne, dédoublonnage
whatsapp	Messages entrants et sortants, sessions de conversation

Module	Responsabilité
ia-vocale	Transcription, extraction, propositions de vente à confirmer
notifications	SMS, push, WhatsApp sortants
tableau-de-bord	Indicateurs agrégés

Ce découpage permet d'extraire plus tard un module en service séparé (par exemple ia-vocale, qui consomme beaucoup de calcul) sans réécrire les autres.

5. Modèle de données

Chaque table métier porte un `boutique_id`, et PostgreSQL bloque lui-même toute lecture hors de la boutique connectée (Row Level Security). Une erreur dans le code ne peut donc pas exposer les données d'un autre commerce.

Règles communes à toutes les tables

- Identifiants UUID v7, générés sur le téléphone : une vente créée hors-ligne a déjà son identifiant définitif.
- Montants en **entiers FCFA** (le franc CFA n'a pas de centimes) : jamais de nombres à virgule pour l'argent.
- Colonnes `created_at`, `updated_at`, `created_by`, `device_id`, `version` pour l'audit et la synchronisation.
- Pas de suppression physique des ventes, factures et paiements : une facture s'annule par un avoir, une dépense par une écriture inverse.

Entité	Attributs clés	Liens
Boutique	nom, raison_sociale, ninea, rccm, adresse, telephone, assujetti_tva, logo, langue	1—N vers tout le reste
Utilisateur	telephone (unique), nom, langue, pin_hash, statut	N—1 Boutique (V1 : un par boutique)
Appareil	utilisateur_id, plateforme, push_token, plage_numeros, derniere_sync	N—1 Utilisateur
Client	nom, telephone, solde_ardoise	1—N Vente

Entité	Attributs clés	Liens
Produit	nom, prix_ht, taux_tva, unite, stock_actuel, seuil_alerte, actif	1—N LigneVente, MouvementStock
Vente	numero_facture, client_id, date, total_ht, total_tva, total_ttc, statut_paiement, source (app, web, whatsapp)	1—N LigneVente, Paiement
LigneVente	produit_id, libelle, quantite, prix_unitaire_ht, taux_tva, total_ligne	N—1 Vente
Paiement	vente_id, montant, methode (especes, wave, orange_money, free_money, carte), statut, reference_paydunya	N—1 Vente
MouvementStock	produit_id, type (vente, reception, ajustement), quantite, reference	N—1 Produit
Depense	date, montant, categorie, libelle, methode, justificatif	N—1 Boutique
PropositionIA	canal, audio_url, transcription, langue, vente_proposee (JSON), statut (en_attente, confirmee, rejetee)	0—1 Vente
EvenementSync	device_id, type, payload, recu_le, traite	journal des lots reçus
Journal	acteur, action, cible, avant, apres, horodatage	audit des actions sensibles

Le livre de caisse n'est pas une table saisie : c'est une vue calculée à partir des Paiements confirmés (entrées) et des Dépenses (sorties). Il ne peut donc jamais diverger des ventes.

Les statuts de paiement d'une vente sont `non_payee`, `payee_en_partie` et `payee`. Le serveur les recalcule à chaque nouveau paiement confirmé ; le solde d'ardoise du client est mis à jour en même temps.

6. Authentification et accès

Le commerçant se connecte avec son numéro de téléphone et un code reçu par SMS. Ensuite, il déverrouille l'app avec un code PIN ou son empreinte : on n'envoie pas un SMS (payant) à chaque ouverture.

Parcours

1. Il saisit son numéro (format +221 vérifié).
2. Le serveur envoie un code à 6 chiffres, valable 5 minutes, par SMS. Le code de secours via WhatsApp (modèle d'authentification) est une option moins chère, à activer si le numéro a WhatsApp.
3. Code correct : le serveur crée la session et enregistre l'appareil. Première connexion : création de la boutique (nom, NINEA, RCCM facultatifs au départ).
4. Le commerçant choisit un PIN à 4 chiffres et peut activer l'empreinte ou Face ID.

Jetons et sessions

- Jeton d'accès (JWT) valable 15 minutes ; jeton de renouvellement valable 30 jours, renouvelé à chaque usage et lié à l'appareil.
- Sur le téléphone, le jeton de renouvellement est rangé dans le coffre sécurisé du système (Keychain sur iOS, Keystore sur Android), débloqué par le PIN ou la biométrie.
- L'empreinte et le visage sont vérifiés par le téléphone lui-même : Kazimara ne reçoit ni ne stocke aucune donnée biométrique.
- Sur le web, la session vit dans un cookie `HttpOnly`, `Secure`, `SameSite=Lax`.

Protection contre les abus et les coûts SMS

- 3 codes maximum par numéro et par heure, limite par adresse IP, 5 essais par code.
- Blocage des préfixes non sénégalais en V1 (fraude aux SMS internationaux).
- Alerte si la dépense SMS quotidienne dépasse un seuil.
- Le PIN bloque l'app après 5 erreurs, puis exige un nouveau code SMS.

Rôles : en V1, un seul rôle `proprietaire` par boutique. Le modèle de données prévoit déjà la table des rôles pour ajouter `gerant`, `vendeur` et `comptable` (lecture seule) en V2 sans migration lourde. La règle des maquettes « seul le gérant peut saisir un prix libre » s'appliquera à ce moment-là.

7. Hors-ligne et synchronisation

Sans réseau, le commerçant peut déclarer une vente, encaisser en espèces, mettre une vente sur l'ardoise, saisir une dépense, et consulter son stock et sa compta. Le paiement mobile money et l'envoi WhatsApp attendent le retour du réseau.

Principe : les ventes sont des événements, jamais des modifications. Une vente, un paiement en espèces, une dépense ou un mouvement de stock s'ajoutent les uns aux autres. Deux ajouts ne peuvent pas entrer en conflit : le serveur les enregistre tous.

Fonctionnement

1. Chaque action est écrite dans SQLite sur le téléphone, avec son UUID, puis ajoutée à une file d'envoi.
2. Dès qu'il y a du réseau, l'app envoie la file par lots de 50 événements maximum (`POST /sync/push`).
3. Le serveur traite chaque événement une seule fois grâce à son UUID : un lot renvoyé deux fois ne crée pas de doublon.
4. L'app récupère ensuite ce qui a changé depuis sa dernière synchronisation (`GET /sync/pull?depuis=<curseur>`) : nouvelles ventes WhatsApp, paiements confirmés, totaux recalculés.
5. Le téléphone garde en local les 90 derniers jours d'historique et tout le catalogue ; le reste se consulte en ligne.

Cas des modifications (fiche produit, prix, client) : la version la plus récente gagne, avec un numéro de version vérifié par le serveur. En V1, avec un seul utilisateur par boutique, ces conflits sont rares.

Stock : le serveur fait le total des mouvements. Si une vente hors-ligne fait passer le stock sous zéro, elle est quand même acceptée (la marchandise est déjà partie) et une alerte « stock à corriger » est levée.

Numérotation des factures hors-ligne

Une facture doit avoir un numéro unique et chronologique, même sans réseau. Le serveur réserve donc à chaque appareil une plage de numéros d'avance (par exemple 100). Le téléphone pioche dans sa plage et en redemande une quand il en reste moins de 20.

- Format : `FAC-2026-000001` , une série par boutique et par année.
- Une plage abandonnée (téléphone perdu) laisse des numéros inutilisés : ils sont inscrits au journal avec leur motif, pour justifier les trous en cas de contrôle.
- Les ventes faites sur le web et par WhatsApp prennent leur numéro directement sur le serveur.
- Le PDF d'une facture hors-ligne est généré sur le téléphone avec la mention « en attente de synchronisation » ; la version définitive avec QR de vérification est produite par le serveur.

8. Paiements via PayDunya

Une facture passe à « Payée » uniquement quand le serveur de PayDunya l'a confirmé, jamais sur la seule parole du téléphone. PayDunya couvre au Sénégal Orange Money, Wave et Free Money (doc PayDunya).

Flux d'un paiement mobile money

1. Le commerçant choisit Wave, Orange Money ou Free Money sur la facture.
2. Le serveur crée une facture de paiement PayDunya, avec en données personnalisées l'identifiant du paiement Kazimara, et l'adresse de notification `POST /webhooks/paydunya`.
3. Le lien de paiement est ouvert sur place ou envoyé au client par WhatsApp.
4. Le client paie. PayDunya appelle la notification de paiement instantanée (IPN) du serveur.
5. Le serveur vérifie la signature de la notification, puis redemande le statut à l'API PayDunya avant d'y croire.
6. Le paiement passe à `confirme`, le statut de la vente et le solde d'ardoise sont recalculés, le commerçant reçoit une notification.

Règles de robustesse

- **Idempotence** : une même notification reçue deux fois ne crée qu'un paiement (clé unique sur la référence PayDunya).
- **Rattrapage** : une tâche vérifie toutes les 15 minutes les paiements restés « en attente » plus de 10 minutes, au cas où une notification se perd.
- **Expiration** : un lien non payé expire après 24 heures.
- **Espèces** : enregistrées directement, sans PayDunya, y compris hors-ligne.
- **Remboursement** : un guide d'intégration indique qu'il n'existait pas d'API publique de remboursement PayDunya en mai 2026 (kolonell) ; à confirmer avec PayDunya. En V1, un remboursement est saisi à la main comme avoir.
- **Frais** : le même guide cite environ 1,7 % de frais moyens, variables selon l'opérateur ; à négocier et vérifier avec PayDunya.

Décision structurante : sur quel compte arrive l'argent ?

Option	Fonctionnement	Avantage	Risque
A. Un compte PayDunya par commerce (recommandé)	Chaque boutique ouvre son compte Business ; Kazimara stocke ses clés chiffrées	L'argent va directement au commerçant ; Kazimara ne touche jamais aux fonds	Onboarding plus long (ouverture du compte)
B. Un compte Kazimara unique	Tout arrive chez Kazimara, qui reverse	Onboarding instantané	Kazimara détient l'argent de tiers : agrément BCEAO probable, responsabilité lourde

L'option A est retenue par défaut. La commission Kazimara sur les transactions (prévue au business plan) se facture alors à part, avec l'abonnement.

9. Facture conforme et vérifiable

La facture V1 porte les mentions attendues au Sénégal et un QR code qui renvoie vers une page de vérification : une banque peut ainsi contrôler qu'elle est authentique. La certification DGID est reportée à plus tard.

Mentions prévues (liste à faire valider par un expert-comptable sénégalais avant la mise en production) :

- Vendeur : raison sociale ou nom commercial, adresse, téléphone, NINEA, RCCM.
- Facture : numéro unique et chronologique, date et heure d'émission.
- Client : nom, et NINEA s'il s'agit d'une entreprise.
- Lignes : désignation, quantité, prix unitaire HT, taux de TVA.
- Totaux : total HT, TVA par taux, total TTC, montant déjà payé, reste à payer, mode de paiement.
- Commerçant non assujetti à la TVA : mention adaptée et aucune TVA facturée (paramètre `assujetti_tva` de la boutique).

Génération

- Le PDF définitif est produit par le serveur (tâche en arrière-plan, bibliothèque `@react-pdf/renderer`), au format A4 et en version ticket 80 mm, puis rangé dans le stockage objet.
- Le partage se fait par un lien signé à durée limitée, ou en pièce jointe WhatsApp.
- Une facture émise n'est jamais modifiée. Une erreur se corrige par un avoir qui l'annule, puis une nouvelle facture.

Page de vérification

Le QR code pointe vers `https://verifier.kazimara.sn/<jeton>` (domaine à réserver). La page affiche le nom du vendeur, le numéro, la date, le total TTC et le statut de paiement, sans aucune autre donnée personnelle. Le jeton est aléatoire et non devinable ; une empreinte SHA-256 du contenu de la facture est conservée pour prouver qu'elle n'a pas été altérée.

10. WhatsApp et IA vocale

Le commerçant peut déclarer une vente en écrivant ou en parlant (français ou wolof) au numéro WhatsApp de Kazimara. L'IA propose la vente, le commerçant la valide d'un bouton, et seulement alors elle est enregistrée. Le wolof est le principal risque technique du projet : il se valide par un test terrain avant le lancement.

Parcours d'une note vocale

1. Le commerçant envoie : « J'ai vendu trois sacs de riz à Fatou, elle a payé par Wave ».
2. Meta transmet le message au serveur (`POST /webhooks/whatsapp`). Le numéro de l'expéditeur identifie la boutique ; un numéro inconnu reçoit un lien d'inscription.
3. L'audio est téléchargé, stocké, puis transcrit par le moteur de reconnaissance vocale (tâche en arrière-plan).
4. Un modèle de langage extrait la vente au format structuré : produits, quantités, client, mode de paiement. Il reçoit le catalogue et la liste des clients de la boutique, pour rattacher « riz » au bon produit et « Fatou » au bon client.
5. Kazimara répond avec un récapitulatif chiffré (prix du catalogue) et trois boutons : Valider, Corriger, Annuler.
6. Sur Valider, la vente est créée comme si elle venait de l'app (numéro de facture, stock, ardoise). Le reçu PDF est renvoyé dans la conversation.
7. Si un élément manque ou est ambigu (produit inconnu, quantité incertaine), Kazimara pose une seule question ciblée au lieu d'inventer.

Pourquoi le wolof demande un test avant de promettre

- Les modèles multilingues généralistes reconnaissent mal le wolof : leurs données d'entraînement en contiennent très peu (arXiv 2601.09716).
- La même étude rapporte, sur le benchmark FLEURS en wolof, environ 41 % de mots erronés pour le meilleur service commercial testé (ElevenLabs Scribe) et 97 % pour Whisper Large v3 non adapté.
- Des modèles affinés sur le wolof existent, dont Kiriku-Wolof-ASR et M-Kiriku d'AI Hub Senegal, et un Whisper affiné sur 51 langues africaines par Sunbird AI (même source).
- Les commerçants mélangent souvent wolof et français dans une même phrase, ce qui complique encore la transcription.

L'étape 4 compense une partie des erreurs : le modèle n'a pas besoin d'une transcription parfaite, seulement de retrouver un produit du catalogue, un nombre et un mode de paiement. La confirmation (étape 5) garantit qu'aucune erreur n'est enregistrée sans être vue.

Choix techniques

- Le moteur de transcription est derrière une interface interne : on peut changer de fournisseur (service commercial, modèle open source auto-hébergé) sans toucher au reste.
- Le français passe par un moteur commercial éprouvé. Pour le wolof, le meilleur moteur est choisi à l'issue du test terrain.
- La messagerie suit les règles de WhatsApp Business : réponse libre dans les 24 heures qui suivent un message du commerçant ; au-delà (relances d'ardoise aux clients), modèles de message pré-approuvés et facturés par Meta.
- Les notes vocales sont conservées 30 jours, pour corriger et améliorer le système, puis supprimées. Le commerçant en est informé à l'inscription.

Test terrain avant le sprint WhatsApp (2 à 3 semaines)

- [ ] Recueillir 200 à 300 vraies notes vocales de ventes auprès de 15 à 20 commerçants à Dakar (avec leur accord).
- [ ] Comparer 2 ou 3 moteurs wolof sur ces enregistrements.
- [ ] Mesurer le taux de ventes correctement extraites de bout en bout, pas seulement la transcription.
- [ ] Fixer le seuil de lancement avec M. Ndiaye (proposition : 85 % de ventes justes avant correction).

11. Contrat d'API

L'API est en REST, versionnée sous `/api/v1`, et décrite intégralement en OpenAPI générée depuis le code NestJS. Le client TypeScript utilisé par le mobile et le web est généré depuis cette description : les deux côtés ne peuvent pas diverger.

Conventions

- La boutique vient toujours du jeton de session, jamais d'un paramètre (`?tenant_id=` du cahier des charges JOKKO est supprimé, c'était une faille potentielle).
- Toute création accepte un en-tête `Idempotency-Key` (l'UUID de l'objet) : une requête rejouée après une coupure ne crée pas de doublon.
- Réponse standard : `{ "data": …, "meta": … }` ; erreur : `{ "error": { "code": "STOCK_INSUFFISANT", "message": "…", "details": … } }`, avec les codes HTTP 400, 401, 403, 404, 409, 422, 429, 500.
- Pagination par curseur ; dates en ISO 8601 UTC ; montants en entiers FCFA.
- Messages d'erreur traduits côté client à partir du `code`.

Méthode	Route	Rôle
POST	/auth/otp/demander	Envoyer un code au numéro
POST	/auth/otp/verifier	Vérifier le code, ouvrir la session
POST	/auth/rafraichir	Renouveler le jeton d'accès
POST	/auth/deconnexion	Fermer la session de l'appareil
GET, PATCH	/boutique	Lire, modifier la fiche commerce
GET, POST, PATCH	/produits	Catalogue
GET, POST, PATCH	/clients	Clients
GET	/clients/{id}/ardoise	Détail de ce que doit un client
GET, POST	/ventes	Lister, créer une vente (+ facture)
GET	/ventes/{id}	Détail d'une vente
GET	/ventes/{id}/pdf	Lien signé vers le PDF
POST	/ventes/{id}/avoir	Annuler par un avoir

Méthode	Route	Rôle
POST	/ventes/{id}/paiements	Enregistrer un paiement en espèces
POST	/ventes/{id}/paiements/mobile-money	Créer un lien de paiement PayDunya
GET, POST	/depenses	Dépenses
POST	/stock/mouvements	Réception, ajustement
GET	/stock/alertes	Produits sous le seuil
GET	/compta/livre-de-caisse	Entrées, sorties, solde par période
GET	/tableau-de-bord	Indicateurs de l'accueil
POST	/sync/push	Envoyer un lot d'événements hors-ligne
GET	/sync/pull	Récupérer les changements depuis un curseur
POST	/appareils/plage-numeros	Réserver une plage de numéros de facture
GET, POST	/propositions-ia/{id}	Lire, valider ou rejeter une vente proposée
POST	/webhooks/paydunya	Notification de paiement (signée)
GET, POST	/webhooks/whatsapp	Vérification et messages entrants Meta (signés)
GET	/public/factures/{jeton}	Page de vérification par QR code

12. Sécurité et protection des données

Héberger en Allemagne est possible techniquement et légalement, mais pas sans formalité : la loi sénégalaise n° 2008-12 soumet le transfert de données personnelles vers un pays tiers à une autorisation préalable de la CDP (Osiris). Le dossier CDP est donc à lancer dès maintenant, quel que soit l'hébergeur retenu.

Hébergement : les deux options

Critère	Allemagne (pilote)	Datacenter au Sénégal
Formalité CDP	Déclaration + autorisation de transfert	Déclaration seule
Délai	Instruction annoncée de 30 à 90 jours par un cabinet (kolonell), à confirmer	Plus court
Cohérence avec le business plan	Contredit la promesse « données hébergées sur le continent »	Conforme
Coût, outillage	Bas, offre cloud mature	Plus élevé, moins de services managés
Latence depuis Dakar	Correcte pour une API	Meilleure

Recommandation : démarrer le pilote en Allemagne si l'autorisation CDP est obtenue à temps, avec une architecture 100 % Docker prête à migrer vers un datacenter sénégalais avant la levée de fonds. À trancher avec M. Ndiaye (section 16).

Mesures de sécurité

- **Transport** : HTTPS (TLS 1.2+) partout, HSTS ; certificats épinglés envisagés pour l'app mobile.
- **Au repos** : disques et sauvegardes chiffrés ; clés PayDunya des commerçants chiffrées champ par champ avec une clé maîtresse hors base.
- **Isolation** : Row Level Security PostgreSQL par boutique, testée automatiquement (un test tente de lire les données d'une autre boutique et doit échouer).
- **Entrées** : toute donnée validée par Zod côté serveur ; requêtes SQL paramétrées par l'ORM ; en-têtes de sécurité sur le web (CSP).
- **Webhooks** : signature vérifiée (PayDunya, Meta), puis contre-vérification du statut par l'API du fournisseur.
- **Secrets** : jamais dans le code ; gestionnaire de secrets de l'hébergeur ou variables chiffrées du CI.
- **Journal d'audit** : connexions, émission et annulation de factures, paiements, exports, changements de prix.
- **Données minimales** : pas de biométrie stockée, pas de pièce d'identité en V1, notes vocales supprimées après 30 jours.

- **Droits des personnes** : export et suppression du compte sur demande, sauf les factures, conservées pour les obligations comptables.

Une relecture juridique (CDP, conservation des factures) et un test d'intrusion externe sont prévus avant l'ouverture au public.

13. Infrastructure, déploiement et supervision

Pour le pilote, trois serveurs suffisent : un pour l'API et les tâches, un pour la base de données, un pour la recette. On ajoute des serveurs d'API quand la charge l'exige, sans rien réécrire.

Environnements

Environnement	Usage	Déploiement
Développement	Poste du développeur, `docker compose` (PostgreSQL, Redis, stockage local)	Local
Recette (staging)	Tests avec PayDunya en mode test, numéro WhatsApp de test, commerçants pilotes	Automatique à chaque fusion sur `main`
Production	Commerçants réels	Manuel, après validation de la recette

Production pilote

- Un serveur applicatif : API NestJS, workers BullMQ, Redis, derrière un proxy (Caddy ou Traefik) qui gère HTTPS.
- Un serveur PostgreSQL dédié, sauvegardes continues.
- Le site Next.js sur le même serveur applicatif ou sur un hébergement de site statique et serveur.
- Stockage objet S3-compatible chez le même hébergeur.
- Objectif de disponibilité : 99,5 % (cahier des charges), soit environ 3 h 40 d'indisponibilité tolérée par mois.

Intégration et livraison continues (GitHub Actions)

1. À chaque demande de fusion : vérification du code, des types et des tests ; refus de la fusion si l'un échoue.
2. À la fusion : construction des images Docker, migrations de base, déploiement en recette.
3. Mise en production sur validation manuelle, avec retour arrière en une commande.
4. Mobile : compilation iOS et Android par EAS Build, publication sur l'App Store et le Play Store. Les corrections qui ne touchent que le code JavaScript partent par mise à jour à distance (EAS Update), sans repasser par les stores.

Sauvegardes

- Archivage continu des journaux PostgreSQL et sauvegarde complète quotidienne, copiés vers un stockage dans une autre région.
- Perte de données maximale visée : 15 minutes. Restauration testée chaque mois sur un serveur vide.
- Rétention : 30 jours de sauvegardes quotidiennes, 12 mois de sauvegardes mensuelles.

Supervision

- Sentry sur le mobile, le web et l'API (erreurs et lenteurs).
- Sonde de disponibilité externe toutes les minutes, alerte SMS ou WhatsApp à l'équipe.
- Tableau de bord technique : temps de réponse (objectif < 300 ms en moyenne), file de tâches, échecs de paiement, coût SMS et IA du jour.

14. Tests et recette

Les calculs d'argent, l'isolation entre boutiques et la synchronisation hors-ligne sont testés automatiquement à chaque changement de code ; la couverture minimale sur la logique métier est de 70 % (cahier des charges).

Niveau	Outil	Ce qui est couvert en priorité
Unitaire	Vitest	TVA, totaux, statuts de paiement, solde d'ardoise, livre de caisse, numérotation
Intégration API	Vitest + Supertest, vraie base PostgreSQL en conteneur	Routes principales, isolation entre boutiques, idempotence, webhooks PayDunya et WhatsApp simulés

Niveau	Outil	Ce qui est couvert en priorité
Bout en bout web	Playwright	Connexion, vente, paiement, dépense
Bout en bout mobile	Maestro	Vente hors-ligne puis synchronisation, connexion OTP, PIN
IA vocale	Jeu d'enregistrements du test terrain	Taux de ventes correctement extraites, rejoué à chaque changement de moteur ou de consigne
Charge	k6	1 800 boutiques simulées au pic de fin de journée, objectif < 300 ms en moyenne

Critères de recette avant ouverture au public

- [ ] Une vente saisie en mode avion apparaît une seule fois sur le serveur après retour du réseau, avec un numéro de facture continu.
- [ ] Un paiement Wave, Orange Money et Free Money réel passe la facture à « Payée » en moins d'une minute.
- [ ] Une notification PayDunya rejouée ne crée pas de second paiement.
- [ ] Aucune requête d'une boutique ne peut lire les données d'une autre.
- [ ] Une vente dictée en wolof et en français est proposée puis enregistrée seulement après validation.
- [ ] Le QR d'une facture ouvre la bonne page de vérification.
- [ ] Restauration complète de la base testée depuis une sauvegarde.
- [ ] L'app tourne correctement sur un Android d'entrée de gamme (2 Go de RAM, Android 8+) en 3G.
- [ ] Recette par 10 à 20 commerçants pilotes à Dakar pendant 2 semaines.

15. Découpage en sprints et coûts

La V1 tient en 10 sprints de 2 semaines, soit environ 5 mois pour un développeur à temps plein épaulé par un second sur le mobile. Les démarches longues (CDP, PayDunya, vérification Meta, comptes stores) démarrent au sprint 0, car elles ne dépendent pas du

code.

Sprint	Semaines	Contenu	En parallèle
0	1–2	Monorepo, CI/CD, environnements, schéma de données validé, OpenAPI initiale	Dossier CDP, compte PayDunya Business, vérification Meta Business, comptes Apple et Google, réservation des domaines
1	3–4	Connexion OTP, PIN, fiche boutique, catalogue produits	Test terrain wolof : collecte des notes vocales
2	5–6	Ventes, factures, numérotation, PDF, page de vérification	Test terrain : comparaison des moteurs
3	7–8	Hors-ligne mobile : SQLite, file d'envoi, synchronisation, plages de numéros	Décision moteur wolof
4	9–10	Encaissements espèces et PayDunya, webhooks, ardoise	—
5	11–12	Stock et alertes, dépenses, livre de caisse, tableau de bord	—
6	13–14	WhatsApp : messages entrants, reçus, relances, liaison au compte	—
7	15–16	IA vocale : transcription, extraction, confirmation, questions de précision	—
8	17–18	App web complète, page d'accueil, traduction anglaise	—
9	19–20	Recette, sécurité, test de charge, pilote 10–20 commerçants, publication stores	Relecture juridique, test d'intrusion

Coûts récurrents de fonctionnement (ordres de grandeur approximatifs pour le pilote, à confirmer par devis)

Poste	Estimation	Remarque
Serveurs, stockage, sauvegardes (Allemagne)	60 à 150 € / mois	3 serveurs + stockage objet
Sentry, sonde de disponibilité	0 à 30 € / mois	Offres gratuites au début
Expo EAS (builds, mises à jour)	0 à 100 $ / mois	Offre gratuite possible au démarrage
Compte Apple Developer	99 $ / an	Obligatoire pour l'App Store
Compte Google Play	25 $ une fois	—
SMS de connexion	Variable, au nombre de SMS	Tarif à obtenir d'un fournisseur sénégalais
WhatsApp Business	Variable, au message	Réponses dans les 24 h incluses ; relances facturées par Meta
IA vocale et modèle de langage	Variable, à la minute d'audio	Chiffré après le test terrain
PayDunya	% par transaction	Payé par le commerçant ou refacturé, à décider
Domaines (.sn, .com)	Quelques dizaines d'€ / an	—

Un hébergement au Sénégal remplacerait la première ligne, pour un coût à obtenir par devis auprès des datacenters locaux.

16. Décisions à prendre avec M. Ndiaye

Cinq décisions conditionnent le sprint 0 ; les autres choix du document sont des valeurs par défaut modifiables.

- [ ] **Hébergement** : Allemagne pour le pilote (avec autorisation CDP de transfert) ou Sénégal dès le départ ?
- [ ] **Compte de paiement** : un compte PayDunya par commerçant (recommandé) ou un compte Kazimara unique ?

- [ ] **Seuil de lancement du vocal wolof** : quel taux de ventes justes après le test terrain (proposition : 85 %) ? Et que fait-on s'il n'est pas atteint : lancement en « bêta » ou report ?
- [ ] **Frais de paiement** : qui paie la commission PayDunya, le commerçant ou Kazimara ?
- [ ] **Validation comptable** : quel expert-comptable sénégalais valide les mentions de la facture et le livre de caisse ?

À mettre à jour dans les documents existants

- Remplacer JOKKO par Kazimara partout ; refaire l'horodatage Fidealis et prévoir le dépôt OAPI au nom Kazimara.
- Maquette de connexion : passer de l'e-mail au numéro de téléphone + code SMS.
- Cahier des charges technique : stack (Expo, Next.js, NestJS), suppression du `tenant_id` en paramètre d'URL, iOS ajouté à Android.
- Business plan, section technique : retirer Supabase et l'hébergement « africain » si l'Allemagne est retenue.

Sources

- PayDunya — documentation développeurs
- Kolonell — intégration PayDunya (2026)
- Opportunities and Challenges of NLP for Low-Resource Senegalese Languages (arXiv 2601.09716)
- Osiris — Dr Mouhamadou Lo sur le transfert des données
- Kolonell — RGPD et loi 2008-12 Sénégal (2026)
- Documents du projet : business plan JOKKO v1.1, cahier des charges, cahier des charges technique v1.0, fiche technique, identité visuelle et maquettes Kazimara.